Classification supervisée et optimisation

OM2-GM01 – Projet Python

Avant de commencer – créer le dépôt Git du projet

Le code et les commentaires du projet seront rendus dans un **dépôt Git partagé par binôme**. L'objectif est de conserver un historique du travail réalisé au cours des différentes séances.

Il est donc demandé d'effectuer des **commits réguliers** au cours du projet, et non un unique commit à la fin.

1. Créer le dossier local et initialiser Git

Un membre du binôme crée un dossier pour le projet, puis initialise le dépôt Git :

```
mkdir projet-python
cd projet-python
git init
git branch -M main
```

Placez ensuite dans ce dossier le notebook fourni pour le projet.

2. Ne pas versionner les fichiers téléchargés automatiquement

Les données Quick, Draw ! sont téléchargées automatiquement par le notebook et stockées dans le dossier `quickdraw_cache`. Il est inutile de les enregistrer dans le dépôt Git.

Créez à la racine du projet un fichier nommé `.gitignore` contenant par exemple :

```
quickdraw_cache/
.ipynb_checkpoints/
__pycache__/
*.pyc
.DS_Store
```

En revanche, le notebook `.ipynb`, les éventuels fichiers `.py`, `.tex` ainsi que le fichier `README.md` doivent être versionnés.

3. Effectuer le premier commit

Une fois les premiers fichiers ajoutés au dossier, enregistrez cet état initial du projet :

```
git add .
git commit -m "Initialisation du projet QuickDraw"
```

4. Créer le dépôt distant

Créez ensuite un dépôt **vide** sur la plateforme GitHub.

Si les fichiers correspondants existent déjà localement, ne demandez pas à la plateforme de créer automatiquement un `README`, un `.gitignore` ou une licence.

Associez ensuite votre dépôt local au dépôt distant, en remplaçant `URL_DU_DEPOT` par l'adresse fournie par la plateforme :

```
git remote add origin URL_DU_DEPOT
git push -u origin main
```

Le membre ayant créé le dépôt distant ajoute ensuite son partenaire comme collaborateur.

Le second membre du binôme peut alors récupérer le projet sur son ordinateur avec :

```
git clone URL_DU_DEPOT
```

Il peut ensuite se placer dans le dossier ainsi créé :

```
cd projet-python
```

5. Travailler avec Git pendant le projet

Avant de commencer à travailler, récupérez les éventuelles modifications déjà présentes sur le dépôt distant :

```
git pull
```

Après avoir terminé une étape cohérente du projet, vérifiez les fichiers modifiés :

```
git status
```

Ajoutez ensuite les modifications, créez un commit et envoyez-le vers le dépôt distant :

```
git add .
git commit -m "Ajout de la descente de gradient a pas fixe"
git push
```

Les messages de commit doivent décrire brièvement le travail effectué. Par exemple :

- `Ajout de l'affichage des données QuickDraw`;
- `Implémentation et test du gradient`;
- `Comparaison des différents pas fixes`;
- `Ajout de la recherche linéaire d'Armijo`;
- `Analyse des erreurs de classification`.

Remarque

Le dépôt Git fait partie du rendu.
L'historique des commits doit permettre de suivre la progression du projet au cours des différentes séances. Évitez donc de travailler pendant plusieurs séances sans effectuer de commit, puis de déposer l'ensemble du notebook en une seule fois à la fin.

Introduction

Le but de ce projet est de construire de bout en bout un programme de classification binaire probabiliste capable de distinguer deux catégories de dessins à main levée.

Les dessins sont transformés en images en niveaux de gris de taille 28×28, puis en vecteurs $x \in \mathbb{R}^{784}$. Après les avoir chargées, vous développerez donc un modèle de régression logistique et déterminerez ses paramètres en résolvant numériquement un problème d'optimisation. Plusieurs méthodes pourront être étudiées et comparées : descente de gradient à pas fixe, recherche linéaire d'Armijo, méthode de Newton ...

Une fois le modèle entraîné, vous étudierez les images correctement et incorrectement reconnues et visualiserez les pixels utilisés par le classifieur pour prendre sa décision. Vous pourrez enfin comparer votre méthode avec les outils de classification disponibles dans scikit-learn et avec un petit réseau de neurones.

À la fin du projet, chaque groupe dessinera et soumettra ses dessins à son classifieur.

1 Trois problèmes au choix

Chaque groupe choisira un des trois couples suivants :

main / pied	pingouin / perroquet	oignon / ananas

Les noms exacts des catégories dans le jeu Quick, Draw ! sont respectivement `hand/foot`, `penguin/parrot` et `onion/pineapple`.

On adoptera la convention suivante : la première catégorie du couple porte l'étiquette $y = 0$ et la seconde l'étiquette $y = 1$.

Question préambule : Commencez par classer intuitivement les trois couples du plus facile au plus difficile pour un humain. Vous reviendrez sur cette réponse à la fin du projet.

2 Générer et afficher les données

Le jeu *Quick, Draw !* contient des dessins à main levée collectés dans le cadre du jeu du même nom. Une image en niveaux de gris codée sur 8 bits par pixel utilise un octet par pixel. Chaque pixel est représenté par une valeur entière comprise entre 0 et 255. L'image peut alors être manipulée comme un tableau NumPy uint8 à deux dimensions : la première dimension correspond à la hauteur et la seconde à la largeur. Les dessins simplifiés peuvent être représentés par des grilles de pixels 28×28.

Ainsi, après normalisation, une image sera transformée en un vecteur

$$x = (x_1, \dots, x_{784}) \in [0,1]^{784}.$$

En régression logistique, nous cherchons à prédire l'étiquette $y \in \{0,1\}$ d'un x non visité, et pour cela, nous utiliserons la probabilité :

$$P(y = 1 \mid x) = \sigma(w^\top x + b),$$

où σ est la fonction sigmoïde $z \mapsto \frac{1}{1+e^{-z}}$. À partir d'une famille de n points $x \in \mathbb{R}^{784}$, associés aux étiquettes connues $(y_i)_{1 \le i \le n} \in \{0,1\}$, nous cherchons à établir la relation entre x et y, de la forme :
- si $\sigma(w^\top x + b) \ge 0.5$, alors $y = 1$,
- sinon $y = 0$,

avec $w = (w^0, w^1) \in \mathbb{R}^2$ et $b \in \mathbb{R}$ les inconnus du problème. Ensuite, pour un nouveau point x, non visité, calculer $\sigma(w^\top x + b) \in [0,1]$ permet de le classer en $y = 0$ ou $y = 1$, tout en étant capable de mesurer l'incertitude de cette prédiction.

Le couple de paramètres (w, b) sera déterminé par un problème d'optimisation : minimiser une fonction de coût, appelée "log-loss", sur les données d'apprentissage.

Pour simplifier le problème et se débarrasser du biais b, nous pouvons augmenter la dimension de x et de w et redéfinir

$$\widetilde{x} = \begin{pmatrix} x \\ 1 \end{pmatrix}, \qquad \theta = \begin{pmatrix} w \\ b \end{pmatrix} \in \mathbb{R}^{785}.$$

Ainsi

$$w^\top x + b = \theta^\top \widetilde{x}.$$

La log-loss sans terme de pénalisation est donné par :

$$f(\theta) = -\frac{1}{n}\sum_{i=1}^{n}(y_i log(\sigma(\theta^T \widetilde{x}_i) + (1-y_i) log(1-\sigma(\theta^T \widetilde{x}_i)).$$

Remarque

Le téléchargement des données Quick, Draw ! et la gestion du cache ne constituent pas le cœur du projet : le code correspondant est donc fourni dans le notebook.
En revanche, la conversion des dessins vectoriels Quick, Draw ! en images exploitables par le classifieur repose sur plusieurs fonctions du module `dessin_process.py` que vous devrez compléter. Ces fonctions sont notamment utilisées par `load_quickdraw_category` lors du chargement initial des données.
En particulier, vous devrez programmer les fonctions permettant de dessiner à l'aide d'un tableau NumPy et de convertir un dessin vectoriel en bitmap 28×28. Une fois ces fonctions complétées, elles seront utilisées automatiquement lors du téléchargement des données.
Les images téléchargées et converties sont ensuite enregistrées dans le dossier `quickdraw_cache`. Lors des exécutions suivantes, elles sont relues directement depuis ce cache et ne sont donc pas téléchargées à nouveau.

2.1 Choix du couple et chargement

La première étape consiste donc à choisir son couple de données entre

main / pied	pingouin / perroquet	oignon / ananas

et à les afficher, en complétant les fonctions du module `dessin_process.py` et en les important. Les fonctions suivantes du module `dessin_process.py` interviennent directement dans la préparation des données :

- `paint_disk` : dessiner un disque dans un tableau NumPy ;
- `paint_segment` : relier deux points par un trait continu ;
- `compute_centering_parameters` : calculer le redimensionnement et le recentrage d'un dessin ;
- `image_converter` : transformer un dessin vectoriel Quick, Draw ! en une image 28×28.

La fonction `load_quickdraw_category`, fournie dans le notebook, appelle ensuite directement `image_converter`.

Dans le notebook, choisissez l'un des couples via la variable `PAIR`. Chargez un nombre identique de dessins pour chaque classe.

Question 2.1. Combien d'observations avez-vous dans chaque classe ? Quelle est la dimension de x avant et après aplatissement des images ? Quelles parties des dessins vous semblent discriminantes ?

Question 2.2. Pourquoi choisit-on ici un nombre identique de dessins dans les deux classes ? Que pourrait-il se passer avec un jeu très déséquilibré ?

2.2 Premières visualisations

Affichez au moins 16 dessins de chaque classe.

Question 2.3. Quelles caractéristiques visuelles vous semblent utiles pour distinguer les deux catégories ?

Question 2.4. Deux dessins d'une même classe peuvent-ils être éloignés dans l'espace des pixels ? Donnez un exemple visuel.

2.3 Images moyennes

Calculez les images moyennes

$$\bar{x}_0 = \frac{1}{n_0} \sum_{i:y_i=0} x_i, \qquad \bar{x}_1 = \frac{1}{n_1} \sum_{i:y_i=1} x_i,$$

et affichez également

$$\bar{x}_1 - \bar{x}_0.$$

Question 2.5. Quelles zones de l'image semblent différencier les deux classes ?

Question 2.6. Pourquoi une image moyenne peut-elle être floue alors que les dessins individuels sont nets ?

Question 2.7. La différence des images moyennes constitue-t-elle déjà un classifieur ? Proposez une règle de décision très simple fondée sur ces moyennes.

2.4 Prétraitement

Transformez les niveaux de gris en réels appartenant à $[0, 1]$, puis aplatissez les images :

$$28 \times 28 \longrightarrow 784.$$

Conservez 70 % des dessins pour l'apprentissage et 30 % pour le test.

Question 2.8. Pourquoi est-il utile de ramener les pixels dans $[0, 1]$ avant d'utiliser une méthode de gradient ?

Question 2.9. Qu'est-ce qui est perdu lorsque l'image est aplatie en un vecteur ?

Question 2.10. Pourquoi le jeu de test ne doit-il pas intervenir pendant l'optimisation ?

L'accuracy, c'est la proportion de prédictions correctes. Avec N exemples, si le modèle en classe correctement N_{correct}, alors accuracy$=\frac{N_{\text{correct}}}{N}$.

Question 2.11. Que mesurerait une accuracy calculée uniquement sur les données d'apprentissage ? Pourquoi cette mesure est-elle insuffisante ?

3 Construire le classifieur !

3.1 Score linéaire et sigmoïde

Pour $x \in \mathbb{R}^{784}$, on considère

$$z = w^\top x + b.$$

Le score z est transformé en probabilité par

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Programmez `sigmoid(z)` et tracez cette fonction.

Question 3.1. Que valent approximativement $\sigma(z)$ pour $z \ll 0$, $z = 0$ et $z \gg 0$?

Question 3.2. Montrez que la règle $\sigma(w^\top x + b) \geq 1/2$ est équivalente à $w^\top x + b \geq 0$.

Question 3.3. Pourquoi une probabilité proche de 0.5 peut-elle être interprétée comme une hésitation du modèle ?

On introduit

$$\widetilde{x} = \begin{pmatrix} x \\ 1 \end{pmatrix}, \qquad \theta = \begin{pmatrix} w \\ b \end{pmatrix} \in \mathbb{R}^{785}.$$

Ainsi

$$w^\top x + b = \theta^\top \widetilde{x}.$$

Question 3.4. Quel avantage pratique apporte cette écriture dans le code ?

3.2 Log-loss

Complétez le code. Programmez `objective(theta, Xb, y, lmbda)` et `grad_objective(...)`.

Pour $p_i = \sigma(\theta^\top \widetilde{x}_i)$, on définit

$$f(\theta) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|w\|^2.$$

Question 3.5. Quelle contribution donne une observation de classe 1 lorsque le modèle prédit $p_i = 0.99$? Et lorsque le modèle prédit $p_i = 0.01$? Comparez qualitativement.

Question 3.6. Pourquoi la log-loss pénalise-t-elle fortement une prédiction très confiante mais fausse ?

Question 3.7. Quel est le rôle de λ ? Que devrait devenir la norme de w lorsque λ augmente ?

Question 3.8. Pourquoi choisit-on ici de ne pas régulariser le biais b ?

3.3 Gradient

On veut montrer que, sans régularisation,

$$\nabla f(\theta) = \frac{1}{n} \widetilde{X}^\top (p - y),$$

et avec régularisation,

$$\nabla f(\theta) = \frac{1}{n} \widetilde{X}^\top (p - y) + \lambda \begin{pmatrix} w \\ 0 \end{pmatrix}.$$

Question 3.9. Retrouvez les formules ci-dessus en utilisant $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Question 3.10. Vérifiez soigneusement les dimensions de toutes les quantités dans l'expression matricielle du gradient.

3.4 Vérification par différences finies

Construisez un gradient numérique par différences centrées :

$$\frac{\partial f}{\partial \theta_j}(\theta) \simeq \frac{f(\theta + h e_j) - f(\theta - h e_j)}{2h}.$$

Comparez-le au gradient analytique pour plusieurs valeurs de h.

Question 3.11. Quel comportement attendez-vous de l'erreur lorsque h diminue ?

Question 3.12. Pourquoi un h extrêmement petit n'est-il pas toujours meilleur en arithmétique flottante ?

Question 3.13. Pourquoi cette vérification est-elle particulièrement utile avant d'implémenter un algorithme d'optimisation ?

4 Optimiser la log-loss ! !

4.1 Descente de gradient à pas fixe

On utilise

$$\theta_{k+1} = \theta_k - \alpha \nabla f(\theta_k).$$

Programmez une fonction retournant : la solution, les valeurs de f, les normes du gradient et le nombre d'itérations.

Question 4.1. Pourquoi $d_k = -\nabla f(\theta_k)$ est-il une direction de descente lorsque le gradient est non nul ?

Question 4.2. Quel critère d'arrêt proposez-vous ? Comparez un critère sur $\|\nabla f\|$ et un critère sur $|f_{k+1} - f_k|$.

Testez plusieurs valeurs de α sur le même point initial, par exemple 10^{-3}, 10^{-2}, 10^{-1}, 0.5, 1, et 2.

Tracez

$$k \mapsto f(\theta_k), \qquad k \mapsto \|\nabla f(\theta_k)\|.$$

Question 4.3. Identifiez au moins un pas donnant une convergence lente et un pas donnant une convergence plus rapide.

Question 4.4. Observez-vous des oscillations ou une divergence pour certains pas ? Expliquez qualitativement.

Question 4.5. Pourquoi le meilleur pas peut-il dépendre du couple de catégories choisi ?

5 Évaluer et comprendre le modèle ! ! !

5.1 Prédictions sur le jeu de test

Calculez les probabilités et les classes sur le jeu de test

$$p_i = \sigma((\theta^\star)^\top \widetilde{x}_i), \qquad \widehat{y}_i = \mathbf{1}_{p_i \geq 0.5}.$$

Afin de connaître les performances de notre classifieur, mesurez, sur les données test, l'accuracy, la précision, le rappel et la matrice de confusion à l'aide du package `sklearn.metrics` et des import de `accuracy_score`, `confusion_matrix`, `precision_score`, `recall_score`.

Indication : L'accuracy est défini plus haut. La précision regarde uniquement la classe 1 et répond à la question suivante : parmi tous les dessins prédits en classe 1, lesquels le sont réellement ?

$$Precision = \frac{VraisPositifs}{VraisPositifs + FauxPositifs}.$$

Le rappel est proche de la précision : parmi tous les dessins de la classe 1, lesquels ont été trouvés par le modèle ?

$$Recall = \frac{VraisPositifs}{VraisPositifs + FauxNégatifs}.$$

La matrice de confusion est de la forme

$$\begin{pmatrix} VN & FP \\ FN & VP \end{pmatrix}$$

avec

VN : classe 0 bien prédite (Vrais Negatifs) ;
FP : classe 0 prédite à tort comme classe 1 ;
FN : classe 1 prédite à tort comme classe 0 ;
VP : classe 1 bien prédite.

Question 5.1. Comparez l'accuracy sur apprentissage et sur test. Une différence importante indique-t-elle nécessairement un sur-apprentissage ?

Question 5.2. Dans votre problème, quelle erreur vous semble la plus gênante : prédire la classe 1 à tort ou manquer un objet de classe 1 ?

Question 5.3. Pourquoi l'accuracy seule peut-elle être trompeuse dans un jeu déséquilibré ?

5.2 Dessins mal classés

Affichez plusieurs faux positifs et faux négatifs, avec leur probabilité prédite.

Question 5.4. Pouvez-vous expliquer visuellement certaines erreurs ?

Question 5.5. Les erreurs proviennent-elles plutôt de dessins intrinsèquement ambigus, de styles inhabituels ou de limites du modèle linéaire ? Donnez des exemples.

Triez les observations selon

$$|p_i - 0.5|.$$

Affichez les dessins pour lesquels cette quantité est la plus petite.

Question 5.6. Les dessins incertains pour le modèle le sont-ils aussi pour vous ?

Question 5.7. Cherchez un exemple où le modèle est très confiant mais se trompe. Pourquoi ce cas est-il plus préoccupant qu'une erreur avec $p \simeq 0.5$?

5.3 Visualiser le vecteur w

Le vecteur $w \in \mathbb{R}^{784}$ peut être remis sous forme d'une image 28×28.

Les pixels associés à des coefficients positifs favorisent la classe 1 ; les coefficients négatifs favorisent la classe 0.

Question 5.8. La carte de poids est-elle facilement interprétable ?

Question 5.9. Comparez w à $\bar{x}_1 - \bar{x}_0$. Quelles ressemblances et quelles différences observez-vous ?

Question 5.10. Pourquoi un coefficient important ne signifie-t-il pas nécessairement que le pixel correspondant est, à lui seul, une caractéristique sémantique de l'objet ?

5.4 Comparaison à sklearn

À la fin seulement, utilisez `sklearn.linear_model.LogisticRegression` comme référence.

Question 5.11. Les performances sont-elles cohérentes avec celles de votre implémentation ? Détaillez.

Question 5.12. Une différence de performance signifie-t-elle nécessairement que l'un des algorithmes d'optimisation est meilleur ? Les deux programmes minimisent-ils exactement la même fonction objectif ?

6 Défi : programmer le dessin et tester votre classifieur

L'objectif est maintenant de tester le classifieur sur vos propres dessins ! Cette dernière partie reprend l'architecture déjà rencontrée dans le TP 2 :

- `dessin_process.py` contient les fonctions de traitement des tableaux NumPy ;
- `dessin_viewer.py` contient l'interface graphique Qt et la gestion des interactions avec l'utilisateur ;
- le notebook conserve le modèle de classification et fournit à l'interface la fonction permettant de classifier un dessin.

Les fonctions `paint_disk` et `paint_segment`, programmées au début du projet dans `dessin_process.py`, sont ainsi réutilisées par l'interface Qt lorsque l'utilisateur dessine à la souris. Elles servent donc à la fois à construire les images Quick, Draw ! et à produire les dessins personnels.

Objectif

Dans cette partie, vous devrez principalement :

- importer et utiliser `prepare_for_model` depuis `dessin_process.py` ;
- compléter la méthode `_paint_at` de `dessin_viewer.py` ;
- compléter la fonction `classify_my_bitmap` dans le notebook ;
- tester le classifieur sur plusieurs dessins personnels et analyser ses réponses.

6.1 Compléter le dessin interactif avec Qt

Le fichier `dessin_viewer.py` contient déjà l'essentiel de l'interface graphique. La classe `DrawingCanvas` possède notamment :

- `self.array`, le bitmap du dessin ;
- `self.drawing`, qui indique si un trait est en cours ;
- `self.last_point`, qui mémorise la dernière position connue de la souris ;
- les méthodes Qt `mousePressEvent`, `mouseMoveEvent` et `mouseReleaseEvent`.

Complétez la méthode `_paint_at(self, x, y)`. Lorsqu'aucun point précédent n'est disponible, elle doit dessiner un disque avec `paint_disk`. Sinon, elle doit relier `self.last_point` au nouveau point avec `paint_segment`. Le point courant devient ensuite le nouveau `self.last_point`.

Question 6.1. Quel est le rôle de `self.last_point` ? Pourquoi est-il nécessaire pour obtenir un trait continu lorsque la souris se déplace rapidement ?

Question 6.2. Pourquoi remet-on `self.last_point` à `None` au début et à la fin d'un trait ?

6.2 Préparer un dessin personnel pour le classifieur

Le modèle a été entraîné sur des images Quick, Draw ! centrées et ramenées à 28×28. Un dessin réalisé dans la fenêtre Qt doit donc subir un prétraitement analogue avant d'être présenté au classifieur.

Importez la fonction `prepare_for_model` depuis `dessin_process.py`. Cette fonction réutilise `compute_centering` déjà employée dans `image_converter`. Les dessins du jeu de données et les dessins personnels suivent ainsi la même convention géométrique.

6.3 Relier l'interface au classifieur

Complétez dans le notebook la fonction

```
def classify_my_bitmap(arr, theta=theta_star):
    img28 = prepare_for_model(arr)
    xb = add_bias(img28.reshape(1, -1))
    p = float(...)
    pred = int(p >= 0.5)

    return {
        "label": class_names[pred],
        "probability": p,
        "class1_name": class_names[1],
    }
```

Cette fonction est passée à l'interface Qt comme fonction de rappel (*callback*). L'interface graphique ne connaît donc pas directement la régression logistique : elle transmet le bitmap au notebook, qui effectue la classification et lui renvoie le résultat.

6.4 Utiliser l'interface Qt

La fenêtre Qt fournie contient trois boutons :

- **Effacer** : remettre le bitmap à zéro ;
- **Classifier** : appliquer `classify_my_bitmap` au dessin courant et afficher la classe ainsi que la probabilité ;
- **Sauver** : enregistrer le dessin courant au format PNG.

Dans le notebook, la fenêtre est créée avec `show_draw`. La commande `%gui qt5` doit avoir été exécutée afin d'intégrer la boucle événementielle Qt à Jupyter.

Pour chaque dessin personnel, le programme doit notamment afficher

$$P(y = 1 \mid x)$$

et la classe prédite.

6.5 Variante : charger un dessin PNG

Si le canevas Qt ne fonctionne pas sur votre environnement, vous pouvez dessiner l'objet dans un autre logiciel, l'enregistrer au format PNG, puis le charger avec `load_png_as_bitmap` depuis `dessin_process.py`.

Cette fonction lit l'image et adopte la convention du projet (fond noir, trait blanc), mais elle ne recentre pas le dessin. Le recentrage est effectué par `prepare_for_model` au moment de la classification.

6.6 Expériences avec votre classifieur

Vous devez tester au moins les situations suivantes :

- ◇ **Expérience 1 — dessins naturels.** Dessinez un objet de chacune des deux classes, sans chercher à aider le modèle.

  Question 6.3. Les deux dessins sont-ils correctement classés ? Avec quelles probabilités ?
- ◇ **Expérience 2 — dessin minimaliste.** Dessinez avec le moins de traits possible tout en restant reconnaissable pour un humain.

  Question 6.4. À partir de quel niveau de simplification le classifieur commence-t-il à hésiter ou à se tromper ?

  Question 6.5. Quelles parties du dessin semblent indispensables pour votre modèle ? Comparez avec la carte des poids w.
- ◇ **Expérience 3 — dessin ambigu.** Produisez volontairement un dessin possédant des caractéristiques des deux classes.

  Question 6.6. Obtenez-vous une probabilité proche de 0.5 ? Votre dessin est-il réellement ambigu pour un humain ou seulement pour le modèle ?
- ◇ **Expérience 4 — tromper le modèle.** Essayez d'obtenir une prédiction incorrecte avec une forte confiance, sans modifier le code ni les paramètres du modèle.

  Question 6.7. Décrivez précisément la stratégie utilisée.

  Question 6.8. Cette expérience met-elle en évidence une limite des données, une limite du modèle linéaire, ou les deux ?

Pour chacune de ces expériences, commentez la prédiction obtenue et demandez-vous si la probabilité annoncée par le modèle est cohérente avec votre propre niveau d'hésitation.

6.7 Bilan du choix initial

Revenez à votre réponse de la question préambule, après avoir comparé avec les autres groupes les accuracies obtenues et les types d'erreurs observés.

Question 6.9. Quel couple semble le mieux séparé par un modèle linéaire ?

7 Bonus

7.1 BONUS Règle d'Armijo

Pour $d_k = -\nabla f(\theta_k)$, on cherche $\alpha > 0$ tel que

$$f(\theta_k + \alpha d_k) \leq f(\theta_k) + c_1 \alpha \nabla f(\theta_k)^\top d_k.$$

Un backtracking part de α_0 et remplace

$$\alpha \leftarrow \rho\alpha, \qquad 0 < \rho < 1,$$

tant que la condition n'est pas satisfaite.

Programmez `armijo_step` puis une descente de gradient utilisant ce pas.

Question 7.1. Pourquoi le membre de droite est-il inférieur à $f(\theta_k)$ lorsque d_k est une direction de descente ?

Question 7.2. Quel est le rôle de c_1 ? Quel est le rôle de ρ ?

Question 7.3. Pourquoi Armijo peut-il être préférable à un pas fixe choisi une fois pour toutes ?

7.2 Comparaison numérique

Comparez pas fixe et Armijo selon :

- nombre d'itérations ;
- nombre d'évaluations de f ;
- temps CPU ;
- valeur finale de f ;
- norme finale du gradient.

Question 7.4. Une méthode utilisant moins d'itérations est-elle nécessairement plus rapide ?

Question 7.5. Quelle méthode recommanderiez-vous pour votre couple de dessins ? Argumentez à partir de vos expériences.

7.3 Règle de Wolfe

Implémentez une recherche linéaire satisfaisant les conditions de Wolfe et comparez-la à Armijo.

7.4 Méthode de Newton

Pour la régression logistique, la Hessienne s'écrit

$$\nabla^2 f(\theta) = \frac{1}{n}\widetilde{X}^\top D \widetilde{X} + \lambda R, \qquad D = \mathrm{diag}(p_i(1-p_i)).$$

Implémentez Newton en résolvant un système linéaire plutôt qu'en calculant explicitement l'inverse.

7.5 Modèle non linéaire

Comparez enfin la régression logistique à un petit réseau de neurones. Toute amélioration devra être mesurée sur le jeu de test, et non sur les données d'apprentissage.

8 Rendu attendu

Code + commentaires sous dépot Git